Building data course — Week 4 — pandas vs SQL vs plain Python: FDD at scale

sparql-playground / lessons

Generated for print and offline reading

Week 4 — pandas vs SQL vs plain Python: FDD at scale

Five daily lessons, all run from Python. Each computation is done twice, as a SQLite query string (stdlib `sqlite3`) and in pandas, and `lab.assert_parity` checks they agree. Run a day with `uv run python lessons/course.py run N` (it uses a temporary copy of `var/timeseries.sqlite`; build that with `uv run bldgts bootstrap`). `assert` lines state the answers you should see. Keep notes under the ignored `student-work/` folder.

Contents

- Day 16 — One answer three ways: a Python loop, pandas and SQL
- Day 17 — Memory: lists, generators, DataFrames and chunks
- Day 18 — Gaps and coverage: sample-and-hold, three ways
- Day 19 — Joining weather: the as-of join
- Day 20 — Scaling showdown, and the Week 4 challenge

Day 16 — One answer three ways: a Python loop, pandas and SQL

Goal: compute the same fault count and run hours with a row-by-row Python loop, with vectorized pandas and with one SQL aggregate; prove they agree; then measure how each one grows with the data, and what moving the data costs.

Read

Week 4 asks one question over and over: *the answer is the same, so which tool should compute it?* You have written Open-FDD-style rules in SQL (Unit 1) and in pandas (Day 14). Plain Python is the third option, and the one a CS course starts with.

Today's job is deliberately small, so the three versions fit on a screen. Take the FC1 raw condition from Day 14 (**fan running and duct static more than 25 Pa below setpoint**) and compute two numbers:

- `fault_rows`: how many 5-minute rows meet the condition;
- `run_hours`: fan run hours, which is rows with `fan_status = 1` times 5 minutes.

How each one does it:

	Plain Python	pandas	SQL (SQLite)
Where the rows live	`sqlite3` cursor or a list	DataFrame columns (NumPy arrays)	table in the database file
Who runs the loop	the interpreter, once per row	compiled C inside NumPy	compiled C inside SQLite
Condition	`if fan == 1 and static < sp - tol`	`(fan == 1) & (static < sp - tol)`	`CASE WHEN ... THEN 1 ELSE 0 END`
Add them up	`count += 1`	`mask.sum()`	`SUM(...)`

Big-O. All three are O(n): they touch every row once, so ten times the rows costs about ten times the time. Big-O hides the **constant factor**, and the constant is what you actually wait for. A Python loop pays interpreter overhead on every row: fetch a tuple, unpack it, run a few bytecodes, update a counter. NumPy and SQLite run the same logic in compiled loops, so their per-row cost is smaller by one to two orders of magnitude. Same curve, very different slope.

The hidden cost: moving data. The pandas and SQL numbers above assume the data is already where that tool wants it: in a DataFrame, or in a table. If your data lives in SQLite and you want pandas, you first pay to copy it into a DataFrame (`read_sql_query`). If it lives in a DataFrame and you want SQL, you pay to insert it (`to_sql`). Those copies are O(n) too, and they are usually *bigger* than the computation they enable. We time them separately.

Run

```
import sqlite3

import numpy as np
import pandas as pd

import lab

conn = (
    lab.connect()
)  # the real Building 50 database; today's benchmarks use throwaway
       in-memory databases

TOL_PA = 25.0  # tolerance below setpoint, pascals
STEP_H = 5 / 60  # one row = 5 minutes, in hours


def make(n):
    """n rows of synthetic SI history for one AHU, in an in-memory
       SQLite table `h` and a DataFrame."""
    df = lab.synthetic_history(n)
    mem = sqlite3.connect(":memory:")
    lab.to_sqlite(df, mem, "h", index=["equipment_id", "ts"])
    return df, mem


df, mem = make(5_000)
print(df.head(3).to_string(index=False))
```

Version (a): a Python loop. Iterating the cursor streams rows out of SQLite one tuple at a time. Nothing but the SQL text and three counters is held in Python.

```
ROWS_SQL = "SELECT fan_status, duct_static, duct_static_sp FROM h"


def loop_answer(conn):
    fault_rows = 0
    run_rows = 0
    for fan, static, sp in conn.execute(ROWS_SQL):
        if fan == 1:
            run_rows += 1
            if static < sp - TOL_PA:
                fault_rows += 1
    return {"fault_rows": fault_rows, "run_hours": run_rows * STEP_H}
```

Version (b): vectorized pandas. No loop in your code. Each line is one operation over a whole column.

```
def pandas_answer(df):
    running = df["fan_status"] == 1
    fault = running & (df["duct_static"] < df["duct_static_sp"] -
         TOL_PA)
    return {"fault_rows": int(fault.sum()), "run_hours":
         float(running.sum()) * STEP_H}
```

Version (c): one SQL aggregate. The loop moves inside the database. Only one row comes back.

```
AGG_SQL = """
SELECT SUM(CASE WHEN fan_status = 1 AND duct_static < duct_static_sp
         - :tol THEN 1 ELSE 0 END) AS fault_rows,
       SUM(fan_status = 1) * :step_h AS run_hours
FROM h
"""


def sql_answer(conn):
    return lab.query(conn, AGG_SQL, {"tol": TOL_PA, "step_h":
         STEP_H})
```

Parity first, as always. Three implementations, one answer:

```
a, b, c = loop_answer(mem), pandas_answer(df), sql_answer(mem)
print("loop  :", a)
print("pandas:", b)
print("sql   :", c.iloc[0].to_dict())
lab.assert_parity(pd.DataFrame([a]), c, label="loop == SQL")
lab.assert_parity(pd.DataFrame([b]), c, label="pandas == SQL")
assert a["fault_rows"] > 0 and a["fault_rows"] == b["fault_rows"] ==
         c.iloc[0].fault_rows
```

`SUM(fan_status = 1)` works because SQLite comparisons evaluate to 0 or 1. And note the cheat in the parity check: the loop and pandas dicts become one-row DataFrames so the same `lab.assert_parity` can compare them with the SQL result.

Timing, growing the data. `lab.best_time` runs a function a few times and keeps the fastest, which hides noise from whatever else your computer is doing. We build the data once per size (outside the timers), then time only the computation. Under `course.py check` the sizes shrink automatically.

```
SIZES = lab.sizes([1_000, 10_000, 100_000, 1_000_000])
rows = []
for n in SIZES:
    df_n, mem_n = make(n)
    reps = 3 if n <= 100_000 else 2
    answers = (loop_answer(mem_n), pandas_answer(df_n),
         sql_answer(mem_n).iloc[0].to_dict())
    assert (
        answers[0]["fault_rows"] == answers[1]["fault_rows"] ==
         answers[2]["fault_rows"]
    )  # parity at every size
    rows.append(
        {
            "rows": n,
            "loop_s": lab.best_time(lambda: loop_answer(mem_n),
         reps),
            "pandas_s": lab.best_time(lambda: pandas_answer(df_n),
         reps),
            "sql_s": lab.best_time(lambda: sql_answer(mem_n), reps),
        }
    )
    mem_n.close()
res = pd.DataFrame(rows)
res["loop/pandas"] = res.loop_s / res.pandas_s
res["loop/sql"] = res.loop_s / res.sql_s
print(res.to_string(index=False, float_format=lambda x: f"{x:,.4g}"))
```

Read the table in two directions. Down a column, the time grows about 10x per row of the table once the data is big enough to matter: that's O(n). (The smallest sizes are dominated by fixed overhead: starting a query, allocating arrays.) Across a row, the three tools differ by a constant factor that stays roughly the same at every size. The loop is the slowest everywhere, and its disadvantage doesn't shrink as the data grows. On the machine this was written on, SQL was about 6x faster than the loop and pandas over 100x faster. Both are compiled loops, but pandas only has to compare and add whole arrays in memory, while SQLite evaluates a `CASE` expression per row, decoding each row from its B-tree pages as it goes.

Sanity-check the growth rather than eyeballing it. Per-row cost should be roughly flat for each tool:

```
per_row = res.assign(
    loop_us=res.loop_s / res.rows * 1e6,
    pandas_us=res.pandas_s / res.rows * 1e6,
    sql_us=res.sql_s / res.rows * 1e6,
)[["rows", "loop_us", "pandas_us", "sql_us"]]
print(per_row.to_string(index=False, float_format=lambda x: f"
         {x:.3f}"))
big = res.iloc[-1]
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert big.loop_s > big.pandas_s and big.loop_s > big.sql_s  #
         the interpreter loop is slowest
```

What did it cost to get the data there? Now the part that surprises people. Time the two copies:

```
move = []
for n in SIZES:
    df_n, mem_n = make(n)
    reps = 3 if n <= 100_000 else 1
    t_read = lab.best_time(
        lambda: pd.read_sql_query("SELECT * FROM h", mem_n), reps
    )  # SQLite -> DataFrame

    def insert():
        scratch = sqlite3.connect(":memory:")
        lab.to_sqlite(df_n, scratch, "h")  # DataFrame -> SQLite
        scratch.close()

    t_insert = lab.best_time(insert, reps)
    t_calc = lab.best_time(lambda: pandas_answer(df_n), reps)
    t_sql = lab.best_time(lambda: sql_answer(mem_n), reps)
    move.append(
        {
            "rows": n,
            "read_sql_s": t_read,
            "to_sql_s": t_insert,
            "pandas_calc_s": t_calc,
            "sql_agg_s": t_sql,
        }
    )
    mem_n.close()
mv = pd.DataFrame(move)
mv["read/calc"] = mv.read_sql_s / mv.pandas_calc_s
mv["load+calc/sql"] = (mv.read_sql_s + mv.pandas_calc_s) /
         mv.sql_agg_s
print(mv.to_string(index=False, float_format=lambda x: f"{x:,.4g}"))
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert (
        mv.read_sql_s > mv.pandas_calc_s
    ).all()  # copying costs more than the arithmetic it feeds
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert (
        mv["load+calc/sql"].iloc[-1] > 1
    )  # from a table, "load then pandas" loses to one SQL aggregate
```

Loading a table into pandas costs about two orders of magnitude more than the pandas computation itself, and inserting a DataFrame into SQLite costs about as much (it builds a B-tree and an index entry per row). The last column is the one to remember: if the data lives in SQLite and you need *one number*, "load into pandas, then compute" loses to the SQL aggregate, because the aggregate runs where the data is and only one row crosses the boundary. "pandas is faster than SQL" is only true *once the data is already a DataFrame*. If you will ask fifty questions of the same data, pay the load once and use pandas. That trade-off is Day 20's main theme.

The worst version of all is the one that looks innocent: loop in Python, but over a list you built with `fetchall()`. You pay the transfer *and* the interpreter, and hold every row in memory as well. Day 17 measures that memory.

Try it

1. The loop above reads three columns. What happens to its time if it reads `SELECT *` (six columns) instead? Time it at 100,000 rows (use 20,000 under quick mode) and say why.
2. Add a third number to all three versions: the *fault run hours*, `fault_rows * 5 / 60`. Keep parity. Which version needed the fewest new lines?

Answers

```
n = 20_000 if lab.QUICK else 100_000
df_n, mem_n = make(n)


def loop_star(conn):
    """Same loop, but SELECT * pulls six columns per row; we unpack
       by position."""
    count = 0
    for _eq, _ts, fan, static, sp, _oat in conn.execute("SELECT *
        FROM h"):
        if fan == 1 and static < sp - TOL_PA:
            count += 1
    return count


t3 = lab.best_time(lambda: loop_answer(mem_n))
t6 = lab.best_time(lambda: loop_star(mem_n))
print(f"3 columns {t3:.3f} s, 6 columns {t6:.3f} s")
assert loop_star(mem_n) == loop_answer(mem_n)["fault_rows"]
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert t6 > t3

# 2. fault hours in all three versions: one new expression each
fault_h_loop = loop_answer(mem_n)["fault_rows"] * STEP_H
fault_h_pd = pandas_answer(df_n)["fault_rows"] * STEP_H
fault_h_sql = lab.query(
    mem_n,
    "SELECT SUM(fan_status = 1 AND duct_static < duct_static_sp -
        :tol) * :step_h AS fault_h FROM h",
    {"tol": TOL_PA, "step_h": STEP_H},
)
lab.assert_parity(pd.DataFrame({"fault_h": [fault_h_loop]}),
        fault_h_sql, label="fault hours loop")
lab.assert_parity(pd.DataFrame({"fault_h": [fault_h_pd]}),
        fault_h_sql, label="fault hours pandas")
print("fault hours:", fault_h_sql.iloc[0, 0])
mem_n.close()
```

1. It gets slower, because the cost per row has a part that depends on the row's width: SQLite has to decode and Python has to build a bigger tuple (and allocate objects for the strings and numbers you never use). Fetch only the columns you need. The same advice applies to `SELECT *` into a DataFrame.
2. The SQL version: it already was one expression, so adding another `SUM` is one line. In pandas it's one line too. The loop needs a new counter *and* a new `if`, and every extra number makes it longer.

Why it matters

Open-FDD ships every rule twice (pandas oracle, SQL pack) because they are better at different jobs, not because one is always right. Today's table is the first honest picture of that. Days 17–20 add the other axes: memory (Day 17), gaps in the data (Day 18), joining another time series (Day 19), and the full scaling showdown (Day 20).

Day 17 — Memory: lists, generators, DataFrames and chunks

Goal: measure how much memory the same query result costs as a list, as a stream and as a DataFrame; shrink the DataFrame with better dtypes; and aggregate a big result in memory proportional to a chunk, not to the table.

Read

Day 16 was about time. The other way a job fails is **memory**: the laptop swaps, or the container is killed. An FDD job over a year of 5-minute data on a hundred AHUs is tens of millions of rows. Three ways to hold a query result:

Shape	What Python holds	Memory grows with
`cursor.fetchall()`	a `list` of tuples, each tuple holding float objects	rows (and a lot of it per row)
`for row in cursor`	one tuple at a time (a **generator**-style stream)	nothing: O(1)
`pd.read_sql_query(...)`	NumPy arrays, one per column (but a list of tuples exists while loading)	rows x bytes per value

A Python float is a 24-byte object, plus an 8-byte pointer in the tuple, plus the tuple's own header. A NumPy `float64` is 8 bytes, period. That's why DataFrames beat lists of tuples for numbers. But the DataFrame is still O(rows), and for text columns it's much worse: an `object` column stores a pointer per row to a separate Python string.

Two memory-saving moves, and one honest warning:

- **Right-size the dtypes.** `equipment_id` has 2 distinct values in 1 million rows: store it as `category` (small integer codes plus one lookup table). A 0/1 status fits in `int8`. `float32` halves a column, but has only about 7 significant digits. That's fine for a temperature, risky for an epoch timestamp, and always something to *check* with parity rather than assume.
- **Stream in chunks.** If the answer is an aggregate (counts, sums, a few rows per equipment), you never need the whole table in memory. Read 50,000 rows, fold them into running totals, throw them away. Peak memory is O(chunk), and the result is the same as one big

aggregate. Because sums and counts are *associative*, summing the chunk sums gives the total.

- **Warning:** `lab.peak_mib` uses `tracemalloc`, which sees memory allocated by Python and by NumPy/pandas buffers. It does **not** see memory SQLite allocates inside its own C library (page cache, sort buffers, the in-memory database itself). A SQL aggregate over a million rows reports about zero because Python really did hold about nothing; SQLite's own footprint just isn't measured here.

Run

```
import sqlite3

import numpy as np
import pandas as pd

import lab

conn = lab.connect()

N = 20_000 if lab.QUICK else 200_000  # rows per AHU; two AHUs
TOL_PA = 25.0

df_src = lab.synthetic_history(N, equips=2)
mem = sqlite3.connect(":memory:")
lab.to_sqlite(df_src, mem, "h", index=["equipment_id", "ts"])
print(f"{len(df_src):,} rows in table h")

ROWS_SQL = "SELECT equipment_id, ts, fan_status, duct_static,
          duct_static_sp FROM h"
```

The task for every version: the count of raw FC1 faults (running, static more than 25 Pa below setpoint) per AHU. First the **reference answer**, one SQL aggregate:

```
AGG_SQL = """
SELECT equipment_id,
       SUM(fan_status = 1 AND duct_static < duct_static_sp - :tol) AS
         fault_rows
FROM h GROUP BY equipment_id ORDER BY equipment_id
"""
ref = lab.query(mem, AGG_SQL, {"tol": TOL_PA})
print(ref.to_string(index=False))
assert (ref.fault_rows > 0).all()
```

Three ways to hold the rows. Each function returns the same per-AHU counts; `lab.peak_mib` reports the peak Python-heap memory while it runs.

```
def counts_fetchall():
    rows = mem.execute(ROWS_SQL).fetchall()  # list of N tuples, all
        alive at once
    out = {}
    for eq, _ts, fan, static, sp in rows:
        out[eq] = out.get(eq, 0) + (1 if fan == 1 and static < sp -
         TOL_PA else 0)
    return out


def counts_stream():
    out = {}
    for eq, _ts, fan, static, sp in mem.execute(ROWS_SQL):  # one
        tuple at a time
        out[eq] = out.get(eq, 0) + (1 if fan == 1 and static < sp -
         TOL_PA else 0)
    return out


def counts_dataframe():
    d = pd.read_sql_query(ROWS_SQL, mem)  # whole result as NumPy
        arrays
    hit = (d.fan_status == 1) & (d.duct_static < d.duct_static_sp -
         TOL_PA)
    return hit.groupby(d.equipment_id).sum().to_dict()


results = {
    "fetchall list": counts_fetchall,
    "cursor stream": counts_stream,
    "DataFrame": counts_dataframe,
}
mem_rows = []
for name, fn in results.items():
    got = fn()
    lab.assert_parity(
        pd.DataFrame({"equipment_id": list(got), "fault_rows":
         list(got.values())}), ref, label=name
    )
    mem_rows.append({"approach": name, "peak_MiB": lab.peak_mib(fn)})
mem_table = pd.DataFrame(mem_rows)
print(mem_table.to_string(index=False, float_format=lambda x: f"
        {x:,.2f}"))
peak = mem_table.set_index("approach").peak_MiB
assert peak["cursor stream"] < peak["fetchall list"] / 20
assert peak["DataFrame"] > peak["cursor stream"] * 20
```

Two surprises. The stream is flat: the loop holds one tuple, so its peak is about zero. And the DataFrame *peaks higher* than the list, even though the finished DataFrame is much smaller than the list. `read_sql_query` has to fetch every row as Python tuples first and then convert them into column arrays, so for a moment both copies exist. The resting size of a DataFrame is small (next), but the peak while loading it is not. That is

exactly what `chunksize` fixes, below. The time cost of the stream is the interpreter loop from Day 16; its memory cost is zero. Streaming trades speed for space.

And the SQL aggregate itself, for contrast. Python holds one 2-row result while SQLite scans every row:

```
sql_peak = lab.peak_mib(lambda: lab.query(mem, AGG_SQL, {"tol":
        TOL_PA}))
print(f"SQL aggregate peak (Python heap only): {sql_peak:.3f} MiB")
assert sql_peak < peak["cursor stream"] + 1
```

That number is real but incomplete: it's the Python side. SQLite's own memory (the page cache, and here the entire in-memory database) is invisible to `tracemalloc`. To see it you would watch the process, for example with `psutil`'s resident set size, rather than the Python allocator.

Shrinking the DataFrame. `memory_usage(deep=True)` counts the real bytes, including the Python objects behind an `object` column. Shallow `memory_usage()` would only count the pointers and badly under-report text.

```
d = pd.read_sql_query(ROWS_SQL, mem)
before = d.memory_usage(deep=True)
print(d.dtypes.to_dict())

small = d.astype(
    {
        "equipment_id": "category",
        "fan_status": "int8",  # a 0/1 flag
        "duct_static": "float32",
        "duct_static_sp": "float32",
    }
)  # ts stays int64: epoch seconds (about 1.8e9) need 32+ bits, and
        float32 would corrupt them
after = small.memory_usage(deep=True)
cmp = pd.DataFrame({"before_MiB": before / 2**20, "after_MiB": after
        / 2**20}).drop(index="Index")
cmp.loc["total"] = cmp.sum()
print(cmp.round(2).to_string())
assert after.sum() < before.sum() * 0.5
assert before["equipment_id"] > 10 * after["equipment_id"]  # text
        object column vs category codes
```

The big win is `equipment_id`: an `object` column of strings becomes a column of 1-byte codes. The float columns halve. Now the check that matters, because `float32` changed the *values*: does the fault answer survive?

```
hit64 = (d.fan_status == 1) & (d.duct_static < d.duct_static_sp -
        TOL_PA)
hit32 = (small.fan_status == 1) & (small.duct_static <
        small.duct_static_sp - np.float32(TOL_PA))
small_counts = hit32.groupby(small.equipment_id, observed=True).sum()
lab.assert_parity(
    small_counts.rename("fault_rows").reset_index(),
    ref,
    label="float32 / int8 / category DataFrame",
)
print("float32 flips:", int((hit64 != hit32).sum()), "rows of",
        len(d))
print("mean static, float64 vs float32:", d.duct_static.mean(),
        small.duct_static.mean())
assert int((hit64 != hit32).sum()) == 0
```

On this data no row flips: the readings carry two decimals and the threshold is a round number, so float32's rounding (about 3e-5 at 350 Pa) never crosses it. On data that sits *exactly on* a threshold, it could. The parity check is how you find out, and why Open-FDD keeps one declared scale and dtype per column instead of guessing.

Chunked reads. The aggregate we want never needs the table in memory at once. `chunksize` makes `read_sql_query` return an iterator of DataFrames; we fold each chunk into running totals. Parity against the SQL aggregate shows nothing was lost.

```
CHUNK = N // 8


def counts_chunked():
    total = pd.Series(dtype="int64")
    for chunk in pd.read_sql_query(ROWS_SQL, mem, chunksize=CHUNK):
        # an iterator of DataFrames
        hit = (chunk.fan_status == 1) & (chunk.duct_static <
         chunk.duct_static_sp - TOL_PA)
        total = total.add(
            hit.groupby(chunk.equipment_id).sum(), fill_value=0
        )  # associative: add the partials
    return total.astype("int64").to_dict()


got = counts_chunked()
lab.assert_parity(
    pd.DataFrame({"equipment_id": list(got), "fault_rows":
        list(got.values())}),
    ref,
    label="chunked",
)
peak_chunked = lab.peak_mib(counts_chunked)
peak_full = peak["DataFrame"]
print(f"full DataFrame {peak_full:.2f} MiB, chunked ({CHUNK:,} rows)
        {peak_chunked:.2f} MiB")
assert peak_chunked < peak_full / 2
```

Memory scales with the **chunk** now: double the table and the peak stays put (only the loop runs longer). The price is that this works for aggregates that can be folded (counts, sums, min/max). A rolling window or a streak that crosses a chunk boundary needs the chunks to carry state across, which is what Day 18's loop does naturally and why streaming a state machine is easy in plain Python.

Try it

1. How big would the full DataFrame be for 100 AHUs x 1 year of 5-minute data (about 105,000 rows each) with five float64 columns plus an int64 `ts`? Compute it from `d.memory_usage`, no new data needed.
2. Make the chunked version also return the *run hours* per AHU (`fan_status == 1` rows x 5 / 60) and prove parity with a SQL aggregate.

Answers

```
# 1. bytes per row of the numeric columns, times rows
numeric_bytes_per_row = 8 * 5 + 8  # five float64 + one int64
rows_total = 100 * 105_000
gib = rows_total * numeric_bytes_per_row / 2**30
print(
    f"{rows_total:,} rows x {numeric_bytes_per_row} B = {gib:.2f} GiB
        (plus the equipment_id column)"
)
assert 0.4 < gib < 0.6


# 2. chunked counts and run hours, folded together
def faults_and_hours():
    total = None
    for chunk in pd.read_sql_query(ROWS_SQL, mem, chunksize=CHUNK):
        running = chunk.fan_status == 1
        part = (
            pd.DataFrame(
                {
                    "fault_rows": (
                        running & (chunk.duct_static <
     chunk.duct_static_sp - TOL_PA)
                    ).astype(int),
                    "run_rows": running.astype(int),
                }
            )
            .groupby(chunk.equipment_id)
            .sum()
        )
        total = part if total is None else total.add(part,
         fill_value=0)
    total["run_hours"] = total.run_rows * 5 / 60
    return total.reset_index()[["equipment_id", "fault_rows",
        "run_hours"]]


sql_both = lab.query(
    mem,
    """SELECT equipment_id, SUM(fan_status = 1 AND duct_static <
        duct_static_sp - :tol) AS fault_rows,
            SUM(fan_status = 1) * 5 / 60.0 AS run_hours FROM h
        GROUP BY equipment_id""",
    {"tol": TOL_PA},
)
lab.assert_parity(
    faults_and_hours(), sql_both, keys=["equipment_id"],
        label="chunked fault rows + run hours"
)
```

1. About 0.5 GiB for 10.5 million rows: fine on a laptop, but tens of GiB across a portfolio of buildings. That is where chunking, dtypes and "let the database aggregate" stop being optional.
2. The partial results are small tables you can add: sums add. A *mean* you'd carry as (sum, count) and divide at the end; a median doesn't fold this way at all.

Why it matters

Memory decides where a rule can run. A rule that needs the whole table as a DataFrame fits a laptop but not an edge controller (Normal Framework on a Raspberry Pi 3 has under 1 GB for everything). Streaming and chunking are what let the same logic run in both places. Day 20 puts time and memory side by side.

Day 18 — Gaps and coverage: sample-and-hold, three ways

Goal: make a fault rule honest when the data has holes: hold each reading until the next one, treat a long gap as "unknown", confirm faults by elapsed time rather than row count, and report coverage next to fault hours. Implement it in SQL, pandas and plain Python, and prove all three agree.

Read

Day 14 counted rows and multiplied by 5 minutes. That works only if every 5-minute row exists. Real controllers go offline, networks drop, a trend log gets cleared. Open-FDD's rules (read `sql/fdd/postgres/rules/cmd1_fan_mismatch.sql`) handle this with **forward sample-and-hold**:

- A reading is assumed to hold until the *next reading arrives*. `held_seconds = next_ts - ts`. A missing row just means its predecessor holds for 10 minutes instead of 5, and the time is still counted.
- If the next reading is more than `MAX_GAP_HOURS` away, we don't know what happened. `held_seconds = 0`: that stretch is **not fault time, not covered time**, and `gap_after = 1` marks the row so the streak **breaks** there. The last row of each equipment has no next reading, so `held_seconds = 0`.
- A fault is **confirmed** when its streak has lasted `CONFIRM_SECONDS` *of elapsed time*: `streak_elapsed = running sum of held_seconds within the streak`.
- Report `fault_hours = sum(held_seconds where confirmed) / 3600` and **coverage**: the seconds we could vouch for divided by the seconds in the window (first to last timestamp).

Why counting rows is wrong once there are gaps. Three things go wrong. (1) A dropped row silently removes 5 minutes of fault time. (2) Two fault rows on either side of a six-hour outage look like consecutive rows, so the streak "confirms" across time you never observed. (3) You cannot tell a healthy machine from one you couldn't see: `fault_hours = 0` with 40% coverage is not good news. Coverage keeps the answer honest.

How each step maps across (compare Day 14's table):

Step	SQL (SQLite)	pandas	plain Python
next timestamp	`LEAD(ts) OVER (PARTITION BY eq ORDER BY ts)`	`df.groupby(eq).ts.shift(-1)`	look at the next row if same equipment
held / gap flag	`CASE WHEN dt > :max_gap_s THEN ...`	`np.where(...)`	`if dt > limit:`
streak breaks	`LAG(raw)` and `LAG(gap_after)` in a `CASE`	`raw.ne(raw.shift())` or previous gap	`if raw != prev_raw or prev_gap:`
elapsed in streak	`SUM(held) OVER (PARTITION BY eq, streak_id ORDER BY ts)`	`held.groupby([eq, streak]).cumsum()`	`elapsed += held`

SQLite has no `EXTRACT(EPOCH FROM ...)` like Postgres, but `ts` is already epoch seconds, so `next_ts - ts` is seconds directly. Units are SI: Pa, and seconds for time.

Run

```
import sqlite3

import numpy as np
import pandas as pd

import lab

conn = lab.connect()

TOL_PA = 25.0
MAX_GAP_HOURS = 1.0
CONFIRM_SECONDS = 600  # 10 minutes, the same as Day 14's two 5-
        minute rows
N = 4_000  # rows per AHU before drops: about 14 days
```

Synthetic data with holes. Two AHUs; 3% of rows dropped at random (short gaps of 10-15 minutes), plus one long outage on AHU_1: 100 rows (about 8 hours) deleted from from inside its weak-fan window, while the fan is on.

```
raw_df = lab.synthetic_history(N, equips=2, gap_prob=0.03)
T0 = int(raw_df.ts.min())
outage = (raw_df.equipment_id == "AHU_1") & raw_df.ts.between(T0 +
         1560 * 300, T0 + 1659 * 300)
gappy = raw_df[~outage].sort_values(["equipment_id",
         "ts"]).reset_index(drop=True)

mem = sqlite3.connect(":memory:")
lab.to_sqlite(gappy, mem, "gap_h", index=["equipment_id", "ts"])
dts = gappy.groupby("equipment_id").ts.diff().dropna()
print(
    f"{len(gappy):,} rows (of {2 * N:,}); step seconds seen:",
    dts.value_counts().sort_index().to_dict(),
)
assert dts.max() > MAX_GAP_HOURS * 3600 and (dts <= 900).sum() > 0.9
         * len(dts)
```

In SQL. Same shape as `cmd1_fan_mismatch.sql`, in SQLite's dialect. One query returns every row's state; a second wraps it to summarize.

```
ROW_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END AS raw_fault
  FROM gap_h
), stepped AS (
  SELECT *, LEAD(ts) OVER (PARTITION BY equipment_id ORDER BY ts) -
         ts AS dt_seconds FROM base
), elapsed_hold AS (
  SELECT *,
         CASE WHEN dt_seconds IS NULL OR dt_seconds <= 0 OR
         dt_seconds > :max_gap_s
              THEN 0.0 ELSE dt_seconds END AS held_seconds,
         CASE WHEN dt_seconds > :max_gap_s THEN 1 ELSE 0 END AS
         gap_after
  FROM stepped
), lagged AS (
  SELECT *,
         CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts)
                   AND COALESCE(LAG(gap_after) OVER (PARTITION BY
         equipment_id ORDER BY ts), 0) = 0
              THEN 0 ELSE 1 END AS is_new_streak
  FROM elapsed_hold
), grp AS (
  SELECT *, SUM(is_new_streak) OVER (PARTITION BY equipment_id ORDER
         BY ts ROWS UNBOUNDED PRECEDING) AS streak_id
  FROM lagged
), ranked AS (
  SELECT *, SUM(held_seconds) OVER (PARTITION BY equipment_id,
         streak_id ORDER BY ts
                                    ROWS UNBOUNDED PRECEDING) AS
         streak_elapsed
  FROM grp
)
SELECT equipment_id, ts, raw_fault, held_seconds, gap_after,
         streak_id, streak_elapsed,
       CASE WHEN raw_fault = 1 AND streak_elapsed >= :confirm_s THEN
         1 ELSE 0 END AS confirmed
FROM ranked
"""
SUMMARY_SQL = (
    """
SELECT equipment_id,
       COUNT(*) AS rows,
       SUM(gap_after) AS long_gaps,
       SUM(CASE WHEN confirmed = 1 THEN held_seconds ELSE 0.0 END) /
         3600.0 AS fault_hours,
       SUM(held_seconds) / 3600.0 AS covered_hours,
       (MAX(ts) - MIN(ts)) / 3600.0 AS window_hours,
       SUM(held_seconds) / (MAX(ts) - MIN(ts)) AS coverage
FROM ("""
    + ROW_SQL
    + """)
GROUP BY equipment_id
"""
```

```
)


def sql_params(max_gap_h=MAX_GAP_HOURS, confirm_s=CONFIRM_SECONDS,
        tol=TOL_PA):
    return {"tol": tol, "max_gap_s": max_gap_h * 3600.0, "confirm_s":
        confirm_s}


rows_sql = lab.query(mem, ROW_SQL, sql_params())
sum_sql = lab.query(mem, SUMMARY_SQL, sql_params())
print(sum_sql.round(3).to_string(index=False))
```

In pandas. Every step a column operation. `shift(-1)` inside a `groupby` is `LEAD`; `shift()` is `LAG`.

```
def gap_pandas(df, max_gap_h=MAX_GAP_HOURS,
      confirm_s=CONFIRM_SECONDS, tol=TOL_PA):
    """df sorted by (equipment_id, ts). Returns the per-row frame
       (same columns as ROW_SQL)."""
    eq = df["equipment_id"]
    raw = ((df.fan_status == 1) & (df.duct_static < df.duct_static_sp
       - tol)).astype(int)
    dt = df.groupby("equipment_id").ts.shift(-1) - df.ts  # NaN on
       each equipment's last row
    limit = max_gap_h * 3600.0
    held = pd.Series(
        np.where((dt > 0) & (dt <= limit), dt, 0.0), index=df.index
    )  # NaN compares False -> 0
    gap_after = (dt > limit).astype(int)
    prev_gap = gap_after.groupby(eq).shift().fillna(0)
    same = (raw == raw.groupby(eq).shift()) & (
        prev_gap == 0
    )  # first row: shift is NaN -> False -> new
    streak_id = (~same).astype(int).groupby(eq).cumsum()
    elapsed = held.groupby([eq, streak_id]).cumsum()
    confirmed = ((raw == 1) & (elapsed >= confirm_s)).astype(int)
    return df[["equipment_id", "ts"]].assign(
        raw_fault=raw,
        held_seconds=held,
        gap_after=gap_after,
        streak_id=streak_id,
        streak_elapsed=elapsed,
        confirmed=confirmed,
    )


def summarize(rows):
    g = rows.assign(fault_s=rows.held_seconds.where(rows.confirmed ==
       1, 0.0)).groupby(
        "equipment_id"
    )
    out = g.agg(
        rows=("ts", "size"),
        long_gaps=("gap_after", "sum"),
        fault_s=("fault_s", "sum"),
        covered_s=("held_seconds", "sum"),
        t0=("ts", "min"),
        t1=("ts", "max"),
    )
    out["fault_hours"] = out.fault_s / 3600.0
    out["covered_hours"] = out.covered_s / 3600.0
    out["window_hours"] = (out.t1 - out.t0) / 3600.0
    out["coverage"] = out.covered_s / (out.t1 - out.t0)
    return out[
        ["rows", "long_gaps", "fault_hours", "covered_hours",
         "window_hours", "coverage"]
    ].reset_index()


rows_pd = gap_pandas(gappy)
```

```
lab.assert_parity(rows_pd, rows_sql, keys=["equipment_id", "ts"],
         label="per row, pandas vs SQL")
sum_pd = summarize(rows_pd)
lab.assert_parity(sum_pd, sum_sql, label="summary, pandas vs SQL")
```

In plain Python. A single forward pass with four pieces of state: the previous raw flag, whether the previous row ended in a gap, the streak number and the running elapsed time. The only look-ahead is the next row's timestamp.

```
def gap_loop(conn, max_gap_h=MAX_GAP_HOURS,
             confirm_s=CONFIRM_SECONDS, tol=TOL_PA):
    data = conn.execute(
        "SELECT equipment_id, ts, fan_status, duct_static,
         duct_static_sp FROM gap_h ORDER BY equipment_id, ts"
    ).fetchall()
    limit = max_gap_h * 3600.0
    out, totals = [], {}
    prev_eq = prev_raw = prev_gap = None
    streak_id = 0
    elapsed = 0.0
    for i, (eq, ts, fan, static, sp) in enumerate(data):
        raw = 1 if fan == 1 and static < sp - tol else 0
        has_next = i + 1 < len(data) and data[i + 1][0] == eq
        dt = data[i + 1][1] - ts if has_next else None
        held = dt if dt is not None and 0 < dt <= limit else 0.0
        gap = 1 if dt is not None and dt > limit else 0
        if eq != prev_eq:  # a new equipment: reset the state machine
            streak_id, elapsed, prev_raw, prev_gap = 0, 0.0, None, 0
            totals[eq] = {"rows": 0, "long_gaps": 0, "fault_s": 0.0,
             "covered_s": 0.0, "t0": ts}
        if raw != prev_raw or prev_gap:
            streak_id += 1
            elapsed = 0.0
        elapsed += held
        confirmed = 1 if raw == 1 and elapsed >= confirm_s else 0
        out.append((eq, ts, raw, held, gap, streak_id, elapsed,
         confirmed))
        t = totals[eq]
        t["rows"] += 1
        t["long_gaps"] += gap
        t["covered_s"] += held
        t["fault_s"] += held if confirmed else 0.0
        t["t1"] = ts
        prev_eq, prev_raw, prev_gap = eq, raw, gap
    rows = pd.DataFrame(
        out,
        columns=[
            "equipment_id",
            "ts",
            "raw_fault",
            "held_seconds",
            "gap_after",
            "streak_id",
            "streak_elapsed",
            "confirmed",
        ],
    )
    summary = pd.DataFrame(
        [
            {
                "equipment_id": eq,
                "rows": t["rows"],
                "long_gaps": t["long_gaps"],
                "fault_hours": t["fault_s"] / 3600.0,
                "covered_hours": t["covered_s"] / 3600.0,
```

```
                "window_hours": (t["t1"] - t["t0"]) / 3600.0,
                "coverage": t["covered_s"] / (t["t1"] - t["t0"]),
            }
            for eq, t in totals.items()
        ]
    )
    return rows, summary


rows_loop, sum_loop = gap_loop(mem)
lab.assert_parity(rows_loop, rows_sql, keys=["equipment_id", "ts"],
        label="per row, loop vs SQL")
lab.assert_parity(sum_loop, sum_sql, label="summary, loop vs SQL")
```

Three implementations, identical on every row and on the summary. Now look at what the gaps did:

```
s = sum_sql.set_index("equipment_id")
print(
    s[["rows", "long_gaps", "fault_hours", "covered_hours",
        "window_hours", "coverage"]]
    .round(3)
    .to_string()
)
assert s.loc["AHU_1", "long_gaps"] == 1 and s.loc["AHU_2",
        "long_gaps"] == 0
assert s.loc["AHU_2", "coverage"] == 1.0  # short drops are held
        over: fully covered
assert s.loc["AHU_1", "coverage"] < 1.0  # the 8-hour outage is not

# covered time = window time minus every gap longer than the limit
        (an independent check)
d_ts = gappy.groupby("equipment_id").ts.diff().dropna()
long_gap_s = d_ts[d_ts > MAX_GAP_HOURS *
        3600].groupby(gappy.equipment_id).sum()
window_s = gappy.groupby("equipment_id").ts.agg(lambda x: x.max() -
        x.min())
assert (
    abs(s.loc["AHU_1", "covered_hours"] * 3600 - (window_s["AHU_1"] -
        long_gap_s["AHU_1"])) < 1e-6
)

print(rows_sql[rows_sql.gap_after == 1].to_string(index=False))
```

Look at the one row with `gap_after = 1`: its `held_seconds` is 0 and the next row starts a new streak, so the fault episode that runs into the outage stops at the outage rather than stretching across it.

Now the wrong way. Day 14's row-counting version, run on the same gappy table:

```
NAIVE_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END AS raw_fault
  FROM gap_h
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts) THEN 0 ELSE 1 END
         AS new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len FROM streaks
)
SELECT equipment_id, SUM(raw_fault = 1 AND streak_len >= 2) * 5 /
         60.0 AS naive_fault_hours
FROM ranked GROUP BY equipment_id
"""
naive = lab.query(mem, NAIVE_SQL, {"tol":
         TOL_PA}).set_index("equipment_id")
cmp = pd.DataFrame({"row_count_hours": naive.naive_fault_hours,
         "gap_aware_hours": s.fault_hours})
cmp["difference_h"] = cmp.gap_aware_hours - cmp.row_count_hours
print(cmp.round(2).to_string())
assert (cmp.difference_h > 0.05).all()  # dropped rows lose time in
         the row-count version
```

AHU_2 has only short drops, and the row-count version is 0.5 hours low: every dropped row took 5 minutes of fault time with it, where sample-and-hold keeps them. AHU_1 is off by much less, but not because the row-count method is right there. It loses time to dropped rows *and* has no notion of the outage (the sample-and-hold version breaks the streak at the outage and counts nothing inside it), so its errors partly cancel by luck. Only the gap-aware numbers come with a coverage figure that tells you how far to trust them.

Try it

1. Raise `MAX_GAP_HOURS` to 12. What happens to AHU_1's coverage, its `gap_after` count and its fault hours? Run all three implementations and check parity.
2. Confirm faults over 1 hour (`CONFIRM_SECONDS = 3600`) instead of 10 minutes. Parity again. Do fault hours rise or fall, and why?

Answers

```
def three_way(max_gap_h, confirm_s, label):
    sq = lab.query(mem, SUMMARY_SQL, sql_params(max_gap_h,
         confirm_s))
    pdf = summarize(gap_pandas(gappy, max_gap_h, confirm_s))
    lp = gap_loop(mem, max_gap_h, confirm_s)[1]
    lab.assert_parity(pdf, sq, label=label + " pandas")
    lab.assert_parity(lp, sq, label=label + " loop")
    return sq.set_index("equipment_id")


# 1. a 12-hour limit makes the outage "held" like any short drop
wide = three_way(12.0, CONFIRM_SECONDS, "12 h limit")
print(wide[["long_gaps", "fault_hours",
          "coverage"]].round(3).to_string())
assert wide.loc["AHU_1", "long_gaps"] == 0 and wide.loc["AHU_1",
          "coverage"] == 1.0

# 2. a one hour confirmation
slow = three_way(MAX_GAP_HOURS, 3600, "1 h confirm")
print(slow[["fault_hours"]].round(2).to_string())
assert (slow.fault_hours < s.fault_hours).all()
```

1. Coverage goes to 1.0 and `gap_after` to 0, and AHU_1 now counts the 8 hours *as if the fault state at the start of the outage persisted through it*. That is the cost of a large `MAX_GAP_HOURS`: you assert knowledge you don't have. Pick the limit from how fast the equipment can change state, not from how much you'd like to cover.
2. Fault hours fall. Faults shorter than an hour never confirm, and in longer ones the first hour of each episode is not counted (a row only counts once its streak's elapsed time reaches the threshold). A longer confirmation time trades sensitivity for fewer false alarms.

Why it matters

Every real Open-FDD rule has this `elapsed_hold` block, and its parity tests check it. A rule without it will pass on clean test data and quietly over- or under-report on a building with a flaky gateway. Day 19 joins a second time series (weather) to these rows, and the same question (how old is too old?) returns.

Day 19 — Joining weather: the as-of join

Goal: attach the most recent weather reading to every 5-minute AHU row, ignoring readings older than two hours, in SQL (correlated subquery plus an index) and in pandas (`merge_asof`); see why the three ways to do it scale so differently; then write a sensor-versus-weather rule on top.

Read

Some Open-FDD rules need a number the building doesn't measure well: the real outdoor temperature. A sensor on an AHU's outside-air intake can sit in the sun, in a shaded louver, or be wired wrong. The **`OAT-METEO`** rule compares it with a weather feed (Open-FDD pulls hourly data from Open-Meteo) and flags a sensor that disagrees with the weather.

The join is awkward because the two series don't share timestamps. The AHU logs every 5 minutes; the weather arrives hourly and sometimes not at all. You can't `JOIN ... ON h.ts = w.ts`. You want, for each AHU row, **the latest weather row at or before that time, but only if it is recent enough** (here: no more than 2 hours old; otherwise it is "stale", and the answer is `NULL`/`NaN`, meaning unknown, not "use whatever's old"). This is an **as-of join**.

The weather in this lesson is a stand-in. There is no weather feed in this repo, so we make one honestly: hourly means of the outside-air temperature, taken from AHU_1's own sensor in synthetic data, plus 0.5 °C of noise (real weather never matches a sensor exactly), with 12% of the hours deleted and one 6-hour block missing, the way a feed with failures looks. In production this table is the Open-Meteo feed. Because it is derived from AHU_1, AHU_1 will agree with it; AHU_2 has a 5 °C sensor offset we inject, which the rule should find.

How the join is done in each tool:

	SQL (SQLite)	pandas	plain Python
Idea	per AHU row, look up the newest weather row <= ts	merge two sorted tables in one pass	per AHU row, search the weather list
Code	`(SELECT w.ts FROM wx w WHERE w.ts <= h.ts AND w.ts >= h.ts`	`pd.merge_asof(h, wx, on="ts", direction="backward", tolerance=...)`	`bisect_right(wx_ts, ts) - 1`

	SQL (SQLite)	pandas	plain Python
	`- :tol ORDER BY w.ts DESC LIMIT 1)`		
Stale handling	extra `AND w.ts >= h.ts - :tol`	`tolerance=` argument	`if ts - wx_ts > tol`

Complexity. Let N be AHU rows and M weather rows.

- A *nested loop* with no index: for each of N rows, scan all M weather rows. **O(N·M)**. SQLite does this when there is no index on `wx(ts)`; so does a naive Python double loop.
- *With an index* (SQLite) or a sorted list and binary search (Python's `bisect`): each lookup is O(log M), so **O(N log M)**.
- `merge_asof` on two *sorted* inputs walks both once: **O(N + M)**. The price: the inputs must be sorted by the key, which costs O(N log N) if they aren't.

Run

```
import sqlite3
from bisect import bisect_right

import numpy as np
import pandas as pd

import lab

conn = lab.connect()

TOL_S = 2 * 3600  # weather older than this is stale
DIFF_C = 3.0  # sensor-vs-weather disagreement, degrees C
CONFIRM_ROWS = 6  # 6 x 5 min = 30 minutes


def make_weather(df, seed=3):
    """A stand-in weather feed: hourly means of df.oat (+ noise),
        with some hours missing."""
    rng = np.random.default_rng(seed)
    hour = (df.ts // 3600) * 3600
    wx = df.groupby(hour).oat.mean().rename("wx_oat").reset_index()
    wx["wx_oat"] = (wx.wx_oat + rng.normal(0, 0.5, len(wx))).round(2)
    wx = wx[rng.random(len(wx)) >= 0.12]
    return wx[~wx.index.isin(range(len(wx) // 2, len(wx) // 2 +
        6))].reset_index(drop=True)


N = 8_640  # 30 days of 5-minute rows per AHU
ahu = lab.synthetic_history(N, equips=2)
a, b = N // 4, N // 4 + N // 5
bias = (
    (ahu.equipment_id == "AHU_2") & (np.arange(len(ahu)) % N >= a) &
        (np.arange(len(ahu)) % N < b)
)
ahu.loc[bias, "oat"] += 5.0  # AHU_2's outside-air sensor reads 5
        degC high for 6 days
wx = make_weather(ahu[ahu.equipment_id == "AHU_1"])
print(f"{len(ahu):,} AHU rows, {len(wx)} weather rows (of {N * 300 //
        3600} hours)")

mem = sqlite3.connect(":memory:")
lab.to_sqlite(ahu, mem, "h", index=["equipment_id", "ts"])
lab.to_sqlite(wx, mem, "wx", index=["ts"])  # indexed
lab.to_sqlite(wx, mem, "wx_plain")  # the same rows, no index
assert len(wx) < N * 300 // 3600
```

The as-of join in SQL. The inner query is run once *per AHU row*, which is what makes it “correlated”. The weather value is then looked up by that timestamp.

```
def join_sql(table="wx"):
    return f"""
SELECT j.equipment_id, j.ts, j.oat, j.fan_status, j.wx_ts, w.wx_oat
FROM (
  SELECT h.equipment_id, h.ts, h.oat, h.fan_status,
         (SELECT w.ts FROM {table} w
          WHERE w.ts <= h.ts AND w.ts >= h.ts - :tol
          ORDER BY w.ts DESC LIMIT 1) AS wx_ts
  FROM h
) j
LEFT JOIN {table} w ON w.ts = j.wx_ts
"""


joined_sql = lab.query(mem, join_sql(), {"tol": TOL_S})
print(joined_sql.head(3).to_string(index=False))
print("rows with no usable weather:",
         int(joined_sql.wx_ts.isna().sum()))
```

The same join in pandas. `merge_asof` needs both sides sorted by the key. Our table has two AHUs interleaved by time, so we sort by `ts` for the merge and sort back afterwards. We copy `ts` into a `wx_ts` column so we can see which weather row matched.

```
def join_pandas(ahu, wx, tol_s=TOL_S):
    left = ahu.sort_values("ts", kind="mergesort")
    right = wx.assign(wx_ts=wx.ts).sort_values("ts")
    out = pd.merge_asof(left, right, on="ts", direction="backward",
          tolerance=tol_s)
    return out[["equipment_id", "ts", "oat", "fan_status", "wx_ts",
          "wx_oat"]]


joined_pd = join_pandas(ahu, wx)
lab.assert_parity(joined_pd, joined_sql, keys=["equipment_id", "ts"],
          label="as-of join")
```

Same rows, same matches, same `NULL`s. (A detail worth knowing: both treat the 2-hour limit as inclusive, so a reading exactly 7,200 s old still counts. The parity check above would have failed if they disagreed.)

How the database does it: `EXPLAIN QUERY PLAN` (Day 12). With the index, each probe is a B-tree range seek; without it, a full scan of the weather table (plus a temporary sort for the `ORDER BY`) per AHU row. (SQLite also builds a throwaway "automatic index" for the final `LEFT JOIN` on `wx_plain`, but not for the subquery's range condition, which is the expensive part.)

```
for table in ("wx", "wx_plain"):
    plan = lab.query(mem, "EXPLAIN QUERY PLAN " + join_sql(table),
         {"tol": TOL_S})
    print(table, "->", plan.detail.tolist())
plan_idx = " ".join(lab.query(mem, "EXPLAIN QUERY PLAN " +
         join_sql("wx"), {"tol": TOL_S}).detail)
plan_plain = " ".join(
    lab.query(mem, "EXPLAIN QUERY PLAN " + join_sql("wx_plain"),
         {"tol": TOL_S}).detail
)
assert "SEARCH w USING COVERING INDEX wx_idx" in plan_idx and "SCAN
         w" in plan_plain
```

Plain Python, two ways. The nested loop is what SQLite does with no index. The `bisect` version is the same idea as the index: keep the weather timestamps sorted and binary-search them.

```
def nested_loop(ts_list, wx_ts_list, tol_s=TOL_S):
    out = []
    for ts in ts_list:
        best = None
        for w in wx_ts_list:  # scans every weather row: O(M) per AHU
         row
            if w <= ts and (best is None or w > best):
                best = w
        out.append(best if best is not None and ts - best <= tol_s
         else None)
    return out


def bisect_loop(ts_list, wx_ts_list, tol_s=TOL_S):
    out = []
    for ts in ts_list:
        i = bisect_right(wx_ts_list, ts) - 1  # O(log M); wx_ts_list
         must be sorted
        out.append(wx_ts_list[i] if i >= 0 and ts - wx_ts_list[i] <=
         tol_s else None)
    return out


ts_list, wx_ts_list = ahu.ts.tolist(), sorted(wx.ts.tolist())
want = joined_sql.set_index(["equipment_id", "ts"]).wx_ts
key = pd.MultiIndex.from_arrays([ahu.equipment_id, ahu.ts])
for name, fn in (("nested loop", nested_loop), ("bisect",
         bisect_loop)):
    got = pd.Series(fn(ts_list, wx_ts_list), index=key,
         dtype="float64")
    lab.assert_parity(
        got.rename("wx_ts").reset_index(),
        want.reset_index(),
        keys=["equipment_id", "ts"],
        label=name,
    )
```

Timing, growing the data. One AHU, weather one hour per 12 AHU rows. We stop the two O(N·M) methods at 10,000 rows; at a million rows they would run for hours. `NaN` means “not run”.

```
def timing_case(n):
    d = lab.synthetic_history(n)
    w = make_weather(d)
    m = sqlite3.connect(":memory:")
    lab.to_sqlite(d[["ts"]], m, "h")  # the timing only needs the
        timestamps
    lab.to_sqlite(w, m, "wx", index=["ts"])
    lab.to_sqlite(w, m, "wx_plain")
    return d, w, m


COUNT_SQL = """
SELECT COUNT(wx_ts) FROM (
  SELECT (SELECT w.ts FROM {t} w WHERE w.ts <= h.ts AND w.ts >= h.ts
        - :tol ORDER BY w.ts DESC LIMIT 1) AS wx_ts
  FROM h)
"""

rows = []
for n in lab.sizes([1_000, 10_000, 100_000, 1_000_000]):
    d, w, m = timing_case(n)
    reps = 3 if n <= 100_000 else 1
    tl, wl = d.ts.tolist(), sorted(w.ts.tolist())
    wx_frame = w.assign(wx_ts=w.ts)
    expected = int(bisect_loop(tl, wl).count(None))
    row = {"N": n, "M": len(w)}
    cases = {
        "sql_index_s": lambda: m.execute(COUNT_SQL.format(t="wx"),
         {"tol": TOL_S}).fetchone()[0],
        "bisect_s": lambda: bisect_loop(tl, wl),
        "asof_s": lambda: pd.merge_asof(
            d[["ts"]], wx_frame, on="ts", direction="backward",
         tolerance=TOL_S
        ),
    }
    if n <= 10_000:  # the O(N*M) methods
        cases["sql_noindex_s"] = lambda: m.execute(
            COUNT_SQL.format(t="wx_plain"), {"tol": TOL_S}
        ).fetchone()[0]
        cases["nested_s"] = lambda: nested_loop(tl, wl)
    for name, fn in cases.items():
        row[name] = lab.best_time(fn, reps)
    matched = int(m.execute(COUNT_SQL.format(t="wx"), {"tol":
         TOL_S}).fetchone()[0])
    assert matched == n - expected  # SQL and bisect found the same
        number of matches
    assert matched == int(cases["asof_s"]().wx_oat.notna().sum())  #
        ... and so did merge_asof
    rows.append(row)
    m.close()
timing = pd.DataFrame(rows)[
    ["N", "M", "nested_s", "sql_noindex_s", "sql_index_s",
        "bisect_s", "asof_s"]
]
print(timing.to_string(index=False, float_format=lambda x: f"
        {x:,.4g}"))
```

Read it in three parts. **The unindexed methods** (`nested_s`, `sql_noindex_s`) grow by about 100x for every 10x of data (N grows 10x and so does M, so N·M grows 100x), which is why they stop at 10,000. SQLite's unindexed subquery is a compiled nested loop, so it beats the Python nested loop by a constant factor but follows the same curve. **With an index** (`sql_index_s`, and `bisect_s`, the Python equivalent) the growth is about 10x per 10x: N lookups, each O(log M). **`merge_asof`** is the quickest because it does the whole job as one O(N + M) merge in compiled code. It has almost no per-row overhead.

The lesson that carries over to all of Week 4: an index (or a sort) changes the *shape* of the curve. Vectorizing only changes the slope.

A sensor-versus-weather rule. In the spirit of `OAT-METEO`: while the fan runs, flag the AHU when its outdoor sensor differs from the weather by more than 3 °C, confirmed for 6 rows (30 minutes). No weather (stale or missing) means *unknown*, so no fault. In SQL, the join result is a CTE under the streak machinery from Day 14.

```
RULE_SQL = (
    "WITH j AS ("
    + join_sql()
    + """), base AS (
  SELECT equipment_id, ts, wx_oat,
         CASE WHEN fan_status = 1 AND wx_oat IS NOT NULL AND ABS(oat
         - wx_oat) > :diff THEN 1 ELSE 0 END AS raw_fault
  FROM j
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts) THEN 0 ELSE 1 END
         AS new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len FROM streaks
)
SELECT equipment_id, ts, wx_oat, raw_fault, streak_id,
       CASE WHEN raw_fault = 1 AND streak_len >= :confirm THEN 1 ELSE
         0 END AS fault
FROM ranked
"""
)
rule_sql = lab.query(mem, RULE_SQL, {"tol": TOL_S, "diff": DIFF_C,
         "confirm": CONFIRM_ROWS})


def rule_pandas(joined, diff_c=DIFF_C, confirm=CONFIRM_ROWS):
    j = joined.sort_values(["equipment_id",
         "ts"]).reset_index(drop=True)
    eq = j.equipment_id
    raw = ((j.fan_status == 1) & j.wx_oat.notna() & ((j.oat -
         j.wx_oat).abs() > diff_c)).astype(int)
    streak =
         raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    length = raw.groupby([eq, streak]).cumcount() + 1
    fault = ((raw == 1) & (length >= confirm)).astype(int)
    return j[["equipment_id", "ts", "wx_oat"]].assign(raw_fault=raw,
         streak_id=streak, fault=fault)


rule_pd = rule_pandas(joined_pd)
lab.assert_parity(rule_pd, rule_sql, keys=["equipment_id", "ts"],
         label="sensor vs weather rule")

hours = rule_sql.groupby("equipment_id").fault.sum() * 5 / 60
print(hours.round(2).to_dict())
assert hours["AHU_2"] > 20 * max(
    hours["AHU_1"], 0.25
)  # the biased sensor is found; the healthy one is quiet
```

AHU_2's offset is found, and AHU_1, which the weather was derived from, stays almost silent. In real data there would be more false alarms, which is why the rule has a 3 °C tolerance, a confirmation time and a stale-

weather guard. Note what the 2-hour tolerance bought: when the weather feed is down, the rule says nothing, instead of comparing a live sensor to yesterday's weather.

Try it

1. Tighten the tolerance to 1 hour. How many AHU rows lose their weather? Check SQL and pandas parity.
2. Replace "latest at or before" with "nearest *after*": SQL `w.ts >= h.ts ... ORDER BY w.ts ASC`, pandas `direction="forward"`. When would you ever want that, and when is it a bug?

Answers

```
# 1. 1-hour tolerance
sql_1h = lab.query(mem, join_sql(), {"tol": 3600})
pd_1h = join_pandas(ahu, wx, 3600)
lab.assert_parity(pd_1h, sql_1h, keys=["equipment_id", "ts"],
        label="1 h tolerance")
lost = int(sql_1h.wx_ts.isna().sum()) -
        int(joined_sql.wx_ts.isna().sum())
print("extra rows without weather:", lost)
assert lost > 0

# 2. forward direction
FORWARD_SQL = (
    join_sql()
    .replace("w.ts <= h.ts AND w.ts >= h.ts - :tol", "w.ts >= h.ts
        AND w.ts <= h.ts + :tol")
    .replace("ORDER BY w.ts DESC", "ORDER BY w.ts ASC")
)
fwd_sql = lab.query(mem, FORWARD_SQL, {"tol": TOL_S})
left = ahu.sort_values("ts", kind="mergesort")
fwd_pd = pd.merge_asof(left, wx.assign(wx_ts=wx.ts), on="ts",
        direction="forward", tolerance=TOL_S)[
    ["equipment_id", "ts", "oat", "fan_status", "wx_ts", "wx_oat"]
]
lab.assert_parity(fwd_pd, fwd_sql, keys=["equipment_id", "ts"],
        label="forward as-of")
assert (fwd_sql.wx_ts.dropna() >=
        fwd_sql.ts[fwd_sql.wx_ts.notna()]).all()
```

1. Rows lose weather in the hours after a feed outage: with a 1-hour limit, a row whose latest reading is 1-2 hours old becomes unknown. The two implementations still agree row for row.
2. The forward join pairs each row with a *future* weather reading. That is legitimate for offline analysis when a value is stamped at the *end* of its interval (a mean over the previous hour). It is a **look-ahead bug** in anything that claims to run live, or in a backtest of a live rule: the rule would "know" something it couldn't have known yet. Always ask which direction is honest for the data you're joining.

Why it matters

“Latest value at or before” is the most common join in building data: setpoints that change a few times a day against readings every minute, schedules against measurements, weather against everything. Remember the three shapes: a nested loop (O(N·M)), an index lookup (O(N log M)), a sorted merge (O(N + M)). Day 20 pits the three tools against each other on the FC1 rule at scale.

Day 20 — Scaling showdown, and the Week 4 challenge

Goal: run Day 14's FC1 confirmation-streak rule as a plain Python state machine, as vectorized pandas and as SQLite window SQL on data that grows 10x per step; compare the times and how they grow; and decide, with numbers, when each one is the right tool.

Read

This is the same rule as Day 14: raw fault = fan running and duct static more than 25 Pa below setpoint; a fault is **confirmed** once the raw fault has held for 2 consecutive rows. All the rule's cleverness is in "consecutive": each row's answer depends on the rows before it. That's the part that makes the tools differ.

	Plain Python	pandas	SQLite
Idea	one pass, a tiny state machine: `streak_len`	column ops: `shift`, `cumsum`, `cumcount`	window functions: `LAG`, `SUM OVER`, `ROW_NUMBER`
State	one integer, carried along	none: streak ids are *recomputed* with `cumsum`	none: window frames recompute it
Work	1 pass, interpreter speed	~5 passes over whole arrays, C speed	sort/partition, several window passes
Memory	O(1) (streams from the cursor)	O(n) (needs all rows in arrays)	the database engine's (not in Python)
Data starts	in the database	has to be loaded first	where it already is

A state machine is the natural way to say "count consecutive rows" in a loop, and the awkward way to say it in column operations (Day 13 needed a trick: compare with the previous row, running-sum the changes). SQL and pandas are both doing that trick. Whether the trick costs more than the interpreter overhead is what the table below shows.

Run

```
import sqlite3

import numpy as np
import pandas as pd

import lab

conn = lab.connect()

TOL_PA = 25.0
CONFIRM = 2  # rows (10 minutes at 5-minute spacing)
```

The three implementations. Each returns two numbers: the count of confirmed fault rows, and the number of episodes (confirmed streaks). The per-row versions exist so we can check every row, not just the totals.

```
ROWS_SQL = "SELECT fan_status, duct_static, duct_static_sp FROM h
          ORDER BY equipment_id, ts"


def loop_rule(conn, keep_rows=False):
    """The state machine: only streak_len survives from one row to
          the next."""
    streak_len, prev_raw = 0, None
    fault_rows = episodes = 0
    flags = []
    for fan, static, sp in conn.execute(ROWS_SQL):  # streams from
          the cursor
        raw = 1 if fan == 1 and static < sp - TOL_PA else 0
        streak_len = streak_len + 1 if raw == prev_raw else 1
        prev_raw = raw
        fault = 1 if raw == 1 and streak_len >= CONFIRM else 0
        fault_rows += fault
        episodes += (
            1 if fault and streak_len == CONFIRM else 0
        )  # the row where a streak first confirms
        if keep_rows:
            flags.append(fault)
    return (fault_rows, episodes, flags) if keep_rows else
          (fault_rows, episodes)


def pandas_rule(h, keep_rows=False):
    """h is sorted by (equipment_id, ts). Day 14's column trick."""
    raw = ((h.fan_status == 1) & (h.duct_static < h.duct_static_sp -
          TOL_PA)).astype(int)
    eq = h.equipment_id
    streak =
          raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    length = raw.groupby([eq, streak]).cumcount() + 1
    fault = ((raw == 1) & (length >= CONFIRM)).astype(int)
    episodes = int(((raw == 1) & (length == CONFIRM)).sum())
    return (int(fault.sum()), episodes, fault) if keep_rows else
          (int(fault.sum()), episodes)


RULE_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END AS raw_fault
  FROM h
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts) THEN 0 ELSE 1 END
         AS new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len FROM streaks
), per_row AS (
```

```
  SELECT equipment_id, ts, streak_id,
         CASE WHEN raw_fault = 1 AND streak_len >= :confirm THEN 1
         ELSE 0 END AS fault
  FROM ranked
)
"""
ROWS_OUT_SQL = RULE_SQL + "SELECT equipment_id, ts, fault FROM
         per_row"
TOTALS_SQL = (
    RULE_SQL + "SELECT COALESCE(SUM(fault), 0) AS fault_rows, "
    "COUNT(DISTINCT CASE WHEN fault = 1 THEN equipment_id || ':' ||
         streak_id END) AS episodes FROM per_row"
)
PARAMS = {"tol": TOL_PA, "confirm": CONFIRM}


def sql_rule(conn):
    r = lab.query(conn, TOTALS_SQL, PARAMS).iloc[0]
    return int(r.fault_rows), int(r.episodes)


def make(n, equips=1):
    df = lab.synthetic_history(n, equips=equips)
    mem = sqlite3.connect(":memory:")
    lab.to_sqlite(df, mem, "h", index=["equipment_id", "ts"])
    return df, mem
```

Parity first, row by row, on a small table with two AHUs (so the `PARTITION BY` matters):

```
df, mem = make(5_000, equips=2)
n_fault, n_eps, flags_loop = loop_rule(mem, keep_rows=True)
_, _, flags_pd = pandas_rule(df, keep_rows=True)
rows_sql = lab.query(mem, ROWS_OUT_SQL, PARAMS)

out_loop = df[["equipment_id", "ts"]].assign(fault=flags_loop)
out_pd = df[["equipment_id", "ts"]].assign(fault=flags_pd)
lab.assert_parity(out_pd, rows_sql, keys=["equipment_id", "ts"],
         label="pandas vs SQL, per row")
lab.assert_parity(out_loop, rows_sql, keys=["equipment_id", "ts"],
         label="loop vs SQL, per row")
print("totals:", loop_rule(mem), pandas_rule(df), sql_rule(mem))
assert loop_rule(mem) == pandas_rule(df) == sql_rule(mem)
assert n_fault > 0 and n_eps > 0
```

Two things to notice in that check. First, `streak_id` restarts at 1 for each AHU (the window is partitioned), so counting episodes across AHUs needs `DISTINCT equipment_id || ':' || streak_id`; counting plain `streak_id` merges different AHUs' episodes into one. Second: the loop above, written for one stream, gave the right answer for two AHUs only because `ORDER BY equipment_id, ts` puts each AHU's rows in a block and the weak-fan window is in the middle of each (the loop would carry a streak from the

end of AHU_1 into the start of AHU_2 otherwise). A real loop needs the same reset logic `PARTITION BY` gives SQL for free (Day 18's loop has it). For the timing below we use one AHU.

The showdown. Synthetic data, 10x more rows each step. For pandas we report two numbers: the computation on a DataFrame already in memory, and "load from SQLite, then compute" (that's the fair comparison if the data lives in the database). Parity of the totals is checked at every size.

```
rows = []
for n in lab.sizes([1_000, 10_000, 100_000, 1_000_000]):
    df_n, mem_n = make(n)
    reps = 3 if n <= 100_000 else 1
    totals = (loop_rule(mem_n), pandas_rule(df_n), sql_rule(mem_n))
    assert totals[0] == totals[1] == totals[2], (n, totals)  # parity
         at every size
    rows.append(
        {
            "rows": n,
            "loop": lab.best_time(lambda: loop_rule(mem_n), reps),
            "pandas": lab.best_time(lambda: pandas_rule(df_n), reps),
            "load+pandas": lab.best_time(
                lambda: pandas_rule(
                    pd.read_sql_query("SELECT * FROM h ORDER BY
         equipment_id, ts", mem_n)
                ),
                reps,
            ),
            "sql": lab.best_time(lambda: sql_rule(mem_n), reps),
        }
    )
    mem_n.close()
t = pd.DataFrame(rows).set_index("rows")
print("seconds")
print(t.map(lambda x: f"{x:,.4g}").to_string())
growth = (t / t.shift(1)).dropna()
print("\ngrowth: time at this size / time at the previous (10x
         smaller) size")
print(growth.round(1).to_string())
```

How to read the growth table: a column of about 10 means O(n): ten times the data, ten times the time. Smaller than 10 at the first rows means fixed overhead (starting a query, allocating arrays) is still a big part of the time. Bigger than 10 would mean worse than linear: sorting adds a log factor, and once arrays stop fitting in CPU cache each pass slows down.

```
big = t.iloc[-1]
print(big.round(3).to_dict())
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert big["loop"] > big["pandas"]  # in-memory column math beats
        the interpreter loop...
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert big["sql"] > big["pandas"]  # ...and beats window SQL, if
        the data is already a DataFrame
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert big["load+pandas"] > big["pandas"]  # but loading is a
        real cost
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert (
        big["loop"] < big["load+pandas"] and big["loop"] < big["sql"]
    )  # from the database, streaming wins here
if not lab.QUICK:  # timings: full sizes only (they vary by machine)
    assert (growth.loc[growth.index[-1]] < 25).all()  # nothing blew
        up: all roughly linear
```

On the machine this was written on, at a million rows (seconds): pandas on resident data 0.44, the streaming Python loop 1.4, load-then-pandas 3.6, window SQL 6.4. Two readings of the same table:

- *If the data is already a DataFrame*, pandas wins by about 3x over the loop. The loop's penalty is smaller than on Day 16's trivial counting because a state machine is *one* pass, while the pandas version needs several full-array passes (`shift`, `ne`, `cumsum`, `groupby.cumcount`).
- *If the data lives in the database*, the honest pandas number is "load, then compute", and that is slower than just streaming the rows through a loop. Window SQL is slowest of all here: SQLite sorts the rows, then makes four window passes, each of which materializes intermediate rows. SQL's strength is not this kind of row-to-row logic. It is the *selective* query that the index answers without reading most of the table (Day 12).

So when does each one win?

- **SQL** when the data already lives in the database and you want a small answer: a total, a top-10, one AHU's last week. The index finds the rows without reading the rest (Day 12), nothing is copied into Python, and the result is one small table. Also when many people or programs need the same logic: the query ships with the data. It loses on cost-per-row for complicated row-to-row logic, and it is awkward to step through.
- **pandas** for in-memory column math and exploratory work: it is the fastest once the data is in arrays, and the interactive loop (change a line, look at a plot) is why it's the oracle in Open-FDD's parity tests. It loses when the data doesn't fit in memory, or when moving the data in costs more than the calculation (Day 16).

- **A plain loop** when the data streams past once and the state is complicated or the data is small: it needs O(1) memory (Day 17), it can **stop early** (try it below), and a state machine with ten fields is easier to read than ten window functions. Its per-row cost is high, but it is flat. If a loop is the right *shape* and merely slow, tools like numba or Cython compile exactly this kind of loop to machine code; that's a story for another course.

Open-FDD ships the pandas version as the reference and the SQL version where the data lives. Both are correct, and the parity test is what lets you pick per situation.

Try it

Stop early. How long does each version take to find the *first* confirmed fault timestamp in a 1,000,000-row table (100,000 under quick mode)? Compare the streaming loop that returns at the first hit with the loop that reads everything. Check that it gets the same timestamp as pandas.

Answer

```
n = 100_000 if lab.QUICK else 1_000_000
df_n, mem_n = make(n)


def first_fault_loop(conn):
    streak_len, prev_raw = 0, None
    for fan, static, sp, ts in conn.execute(
        "SELECT fan_status, duct_static, duct_static_sp, ts FROM h
         ORDER BY equipment_id, ts"
    ):
        raw = 1 if fan == 1 and static < sp - TOL_PA else 0
        streak_len = streak_len + 1 if raw == prev_raw else 1
        prev_raw = raw
        if raw == 1 and streak_len >= CONFIRM:
            return ts  # early exit: the rest of the table is never
         read
    return None


def first_fault_pandas(h):
    *_, fault = pandas_rule(h, keep_rows=True)
    return int(h.ts[fault == 1].iloc[0])


assert first_fault_loop(mem_n) == first_fault_pandas(df_n)
t_early = lab.best_time(lambda: first_fault_loop(mem_n))
t_full = lab.best_time(lambda: loop_rule(mem_n))
t_pd = lab.best_time(lambda: first_fault_pandas(df_n))
print(f"loop with early exit {t_early:.3f} s, full loop {t_full:.3f}
        s, pandas {t_pd:.3f} s")
assert t_early < t_full * 0.7
```

The early-exit loop does a fraction of the work (and `ORDER BY equipment_id, ts` matters: it matches the index, so SQLite streams rows in order instead of sorting a million of them first), because the first confirmed fault is early in the table (the weak fan window starts a third of the way in, and before that an occasional noise streak trips it). pandas and the window SQL have no equivalent: they compute every row's state before you can ask for the first. A `LIMIT 1` on the SQL doesn't help for the same reason. Early exit is the loop's unique strength: here it takes the loop from 1.3 s to 0.5 s, and it would matter far more if the first fault were near the start or each row cost a network call. (pandas, with everything in memory, can still be faster in raw time; the loop's win is in *not needing the rest of the data at all*.)

Week 4 challenge

Find the **longest confirmed fault episode** for each AHU: its length in rows and in minutes, on a table with two AHUs of 20,000 rows each (5,000 under quick mode). An episode is a maximal run of consecutive raw-fault rows that lasts at least `CONFIRM` rows. Implement it three ways (a streaming loop that tracks the current and best run, a pandas `groupby` on the streak id, and SQL `GROUP BY` the streak id), check parity of all three, and say which one you would ship if the data were (a) one building's database, (b) a notebook of a month of data, (c) a stream on a controller.

Answer

```
n = 5_000 if lab.QUICK else 20_000
df2, mem2 = make(n, equips=2)

# (a) streaming loop: per-equipment state, reset whenever the
        equipment changes
best = {}
cur_eq = prev_raw = None
run = 0
for eq, fan, static, sp in mem2.execute(
    "SELECT equipment_id, fan_status, duct_static, duct_static_sp
        FROM h ORDER BY equipment_id, ts"
):
    raw = 1 if fan == 1 and static < sp - TOL_PA else 0
    if eq != cur_eq:
        cur_eq, prev_raw, run = eq, None, 0
        best.setdefault(eq, 0)
    run = run + 1 if raw == prev_raw else 1
    prev_raw = raw
    if raw == 1 and run >= CONFIRM:
        best[eq] = max(best[eq], run)
loop_df = pd.DataFrame({"equipment_id": list(best), "longest_rows":
        list(best.values())})
loop_df["longest_min"] = loop_df.longest_rows * 5.0

# (b) pandas: size of each raw-fault streak, keep those that confirm,
        take the max per AHU
raw = ((df2.fan_status == 1) & (df2.duct_static < df2.duct_static_sp
        - TOL_PA)).astype(int)
eq = df2.equipment_id
streak = raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
sizes = raw.groupby([eq, streak]).agg(["first", "size"])
ep = sizes[(sizes["first"] == 1) & (sizes["size"] >= CONFIRM)]
        ["size"]
pd_df = ep.groupby("equipment_id").max().rename("longest_rows").reset_index()
pd_df["longest_min"] = pd_df.longest_rows * 5.0

# (c) SQL: GROUP BY the streak id, filter, then MAX
LONGEST_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END AS raw_fault
  FROM h
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts) THEN 0 ELSE 1 END
         AS new_streak FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), episodes AS (
  SELECT equipment_id, streak_id, COUNT(*) AS rows_in_streak
  FROM streaks WHERE raw_fault = 1 GROUP BY equipment_id, streak_id
         HAVING COUNT(*) >= :confirm
)
SELECT equipment_id, MAX(rows_in_streak) AS longest_rows,
         MAX(rows_in_streak) * 5.0 AS longest_min
```

```
FROM episodes GROUP BY equipment_id
"""
sql_df = lab.query(mem2, LONGEST_SQL, PARAMS)
print(sql_df.to_string(index=False))

lab.assert_parity(pd_df, sql_df, keys=["equipment_id"],
         label="longest episode, pandas vs SQL")
lab.assert_parity(loop_df, sql_df, keys=["equipment_id"],
         label="longest episode, loop vs SQL")
assert (sql_df.longest_rows >= CONFIRM).all() and len(sql_df) == 2
```

Which to ship: (a) one building's database: **SQL**, next to the data, with an index, returning two numbers. (b) a notebook with a month of data: **pandas**, because you'll want to plot the episodes next. (c) a stream on a controller: the **loop**, with O(1) memory and early updates the moment each episode ends. All three give the same answer, which is the point of having the parity check.

Week 4 in one sentence: the answer is the same everywhere, and *where the data is, how much of it you hold in memory, and what shape the logic has* decide which tool gets there cheapest.

Reading — Week 4

The lessons first, then these for exact behavior.

- Python timeit
- Python tracemalloc
- pandas: scaling to large datasets
- pandas.merge_asof
- SQLite query optimizer overview
- Open-FDD SQL cookbook